

ML4T ETF Rotation Project

Claude Code Reference Document — Full Project Specification

Project Goal

Create a machine learning-driven multi-asset ETF rotation strategy and determine whether it would be more effective than Faber's simple momentum-based rotation strategy (10-month SMA rule) used as a baseline.

Background & Motivation

Based on Mebane T. Faber's 2006 paper 'A Quantitative Approach to Tactical Asset Allocation'. Faber proposed:

- Buy when monthly price > 10-month simple moving average (SMA)
- Sell and move to cash when monthly price < 10-month SMA
- Same rule applied equally across all five asset classes

Faber's three design criteria:

1. Simple, non-optimized, purely mechanical logic
2. The same model and parameters for every asset class
3. Price-based only

Limitations of Faber's approach:

- No single timeframe works best — 10-month SMA may miss signals
- Position sizing not addressed — equal weighting ignores volatility differences
- No macro context considered — only uses price data
- Multiple rotations can be expensive in transaction costs

ML Solution

LightGBM ranking model addresses all four limitations:

- Takes multiple timeframes into consideration simultaneously
- Combined with risk parity weighting to account for ETF volatility
- Incorporates macro context alongside price data
- Learns the optimal rotation strategy that minimizes unnecessary trades

ETF Universe

Five ETFs equivalent to Faber's five asset classes:

Ticker	Name	Faber Equivalent	Inception Date	AUM
SPY	SPDR S&P 500 ETF Trust	S&P 500	01/22/1993	\$652B
EFA	iShares MSCI EAFE ETF	MSCI EAFE	08/14/2001	\$72B
TLT	iShares 20+ Year Treasury Bond ETF	US 10-Year Bonds	07/22/2002	\$42B
VNQ	Vanguard Real Estate ETF	NAREIT	09/23/2004	\$34B
DBC	Invesco DB Commodity Index Fund	GSCI	02/03/2006	\$1.3B

Note: DBC is the limiting factor. Backtest start date is February 2007 (DBC inception + 12 months for momentum warmup).

Backtest Period

- Start date: February 2007
- End date: December 2024
- Total period: approximately 18 years
- Both strategies (Faber baseline and ML) must run over the exact same period on the exact same ETFs.

Macro Indicators (via ALFRED)

Use ALFRED (Archival FRED) instead of regular FRED to avoid lookahead bias from data revisions. Access via fredapi with realtime_start and realtime_end parameters.

FRED Series ID	Indicator	Why It Matters
FEDFUNDS	Federal Funds Rate	Affects bond prices and equity valuations
CPIAUCSL	Consumer Price Index	Inflation — benefits commodities and real estate
T10Y2Y	10Y minus 2Y Treasury Spread	Yield curve — signals recession risk
INDPRO	Industrial Production Index	Economic growth — favors equities and commodities

Project Phases

Phase I: Data Collection & Cleaning

What to code:

- Download daily adjusted closing prices for SPY, EFA, TLT, VNQ, DBC using yfinance
- Download FEDFUNDS, CPIAUCSL, T10Y2Y, INDPRO from ALFRED using fredapi with vintage dates
- Forward fill missing values on non-trading days
- Align all data to the same monthly date index

- Check for and handle outliers
- Save cleaned ETF prices and macro indicators to disk as separate CSV or parquet files

Key code snippets:

```
import yfinance as yf
from fredapi import Fred
ETFs = ['SPY', 'EFA', 'TLT', 'VNQ', 'DBC']
START = '2006-02-01' # 1 year before backtest start for warmup
END = '2024-12-31'
# For ALFRED point-in-time data:
fred = Fred(api_key='YOUR_API_KEY')
fedfunds = fred.get_series_all_releases('FEDFUNDS')
```

Output: cleaned_prices.csv and cleaned_macro.csv saved to disk

Phase II: EDA & Feature Engineering

What to code:

- Plot price charts for each ETF over full history
- Plot return distributions for each ETF
- Plot rolling correlation heatmap between ETFs
- Calculate momentum signals at each month end for each ETF:
 - 1-month: (price today - price 1 month ago) / price 1 month ago
 - 3-month, 6-month, 12-month: same formula
- Calculate rolling 60-day volatility (std dev of daily returns)
- Align macro indicators to month end dates using forward fill
- Save feature table to disk

Output: feature_table.csv where each row = one ETF at one month end, columns = all signals + macro indicators

Phase III: Running Tests

Part A — Faber Baseline:

- Calculate 10-month SMA for each ETF at each month end
- If price > 10-month SMA: invest. If price < SMA: move to cash
- Equal weight across all invested ETFs
- Run monthly rebalancing loop from Feb 2007 to Dec 2024
- Record monthly portfolio returns

Part B — ML Strategy:

- Set up walk-forward validation windows (e.g. 3 years train, 1 year test, roll forward)
- Label training data by ranking ETFs from best to worst actual next-month performance

- Train LightGBM ranking model (lambdarank) on feature table
- At each month end: generate ETF rankings using only data available at that time
- Apply risk parity weighting: $\text{weight} = (1/\text{volatility}) / \sum(1/\text{volatility for all ETFs})$
- Run monthly rebalancing loop over same period as baseline
- Record monthly portfolio returns

CRITICAL: Never use future data to make past decisions. Walk-forward validation enforces this.

Phase IV: Comparison & Evaluation

What to code:

- Calculate CAGR, Sharpe ratio, maximum drawdown, and standard deviation for both strategies
- Plot cumulative returns of both strategies on the same chart
- Generate full performance report using pyfolio
- Produce side by side comparison table

Metric	Faber Baseline	ML Strategy
CAGR	?	?
Sharpe Ratio	?	?
Max Drawdown	?	?
Std Deviation	?	?

Key Formulas

Momentum: $(\text{Price today} - \text{Price N months ago}) / \text{Price N months ago}$

Volatility: Rolling standard deviation of daily returns over 60 days

Risk Parity Weight: $(1 / \text{volatility}) / \sum(1 / \text{volatility for all ETFs})$

Sharpe Ratio: $(\text{Average monthly return} - \text{risk free rate}) / \text{std dev of monthly returns}$

Max Drawdown: $(\text{Peak portfolio value} - \text{Trough portfolio value}) / \text{Peak portfolio value}$

Required Libraries

Library	Purpose
yfinance	Download ETF historical price data from Yahoo Finance
fredapi	Download macro indicators from ALFRED (point-in-time)
pandas	Data manipulation and alignment
numpy	Numerical calculations
lightgbm	LightGBM ranking model training

scikit-learn	Walk-forward validation setup
matplotlib / seaborn	Plotting and visualization
pyfolio	Portfolio performance evaluation and reporting

Important Notes for Implementation

Lookahead Bias

This is the most critical thing to avoid. At every point in the simulation you can **ONLY** use data that existed before that date. Walk-forward validation enforces this for the ML model. For the Faber baseline, make sure the SMA is calculated using only past prices.

ALFRED vs FRED

Do **NOT** use regular FRED — it shows revised data that was not available at the time. Always use ALFRED with `realtime_start` and `realtime_end` parameters to get point-in-time vintage data.

ETF Selection Bias

The five ETFs were selected based on their availability at the backtest start date (Feb 2007), not based on current performance. DBC was the last to launch (Feb 2006) which determined the start date.

Walk-Forward Validation

Suggested windows: train on 3 years of data, test on 1 year, then roll forward 1 year and repeat. This simulates real-world performance without leaking future data into training.

8-Week Timeline

Week	Phase	Tasks
1-2	Phase I	Data collection and cleaning — yfinance + ALFRED
3	Phase II	EDA and feature engineering
4	Phase III-A	Implement and backtest Faber baseline
5-6	Phase III-B	Train LightGBM model and backtest ML strategy
7	Phase IV	Comparison, evaluation, and charts
8	Buffer	Report writing and debugging